

Deployability (Desplegabilidad)

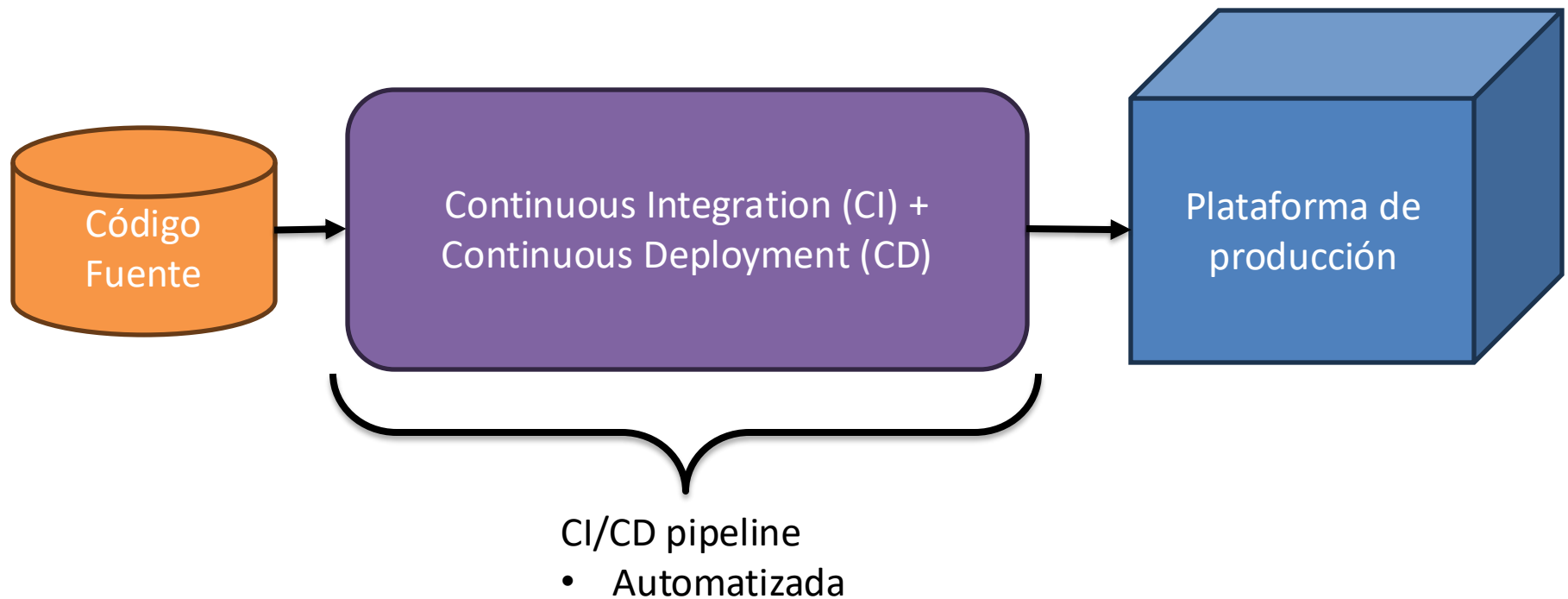
En software, ¿Qué es un **release**?



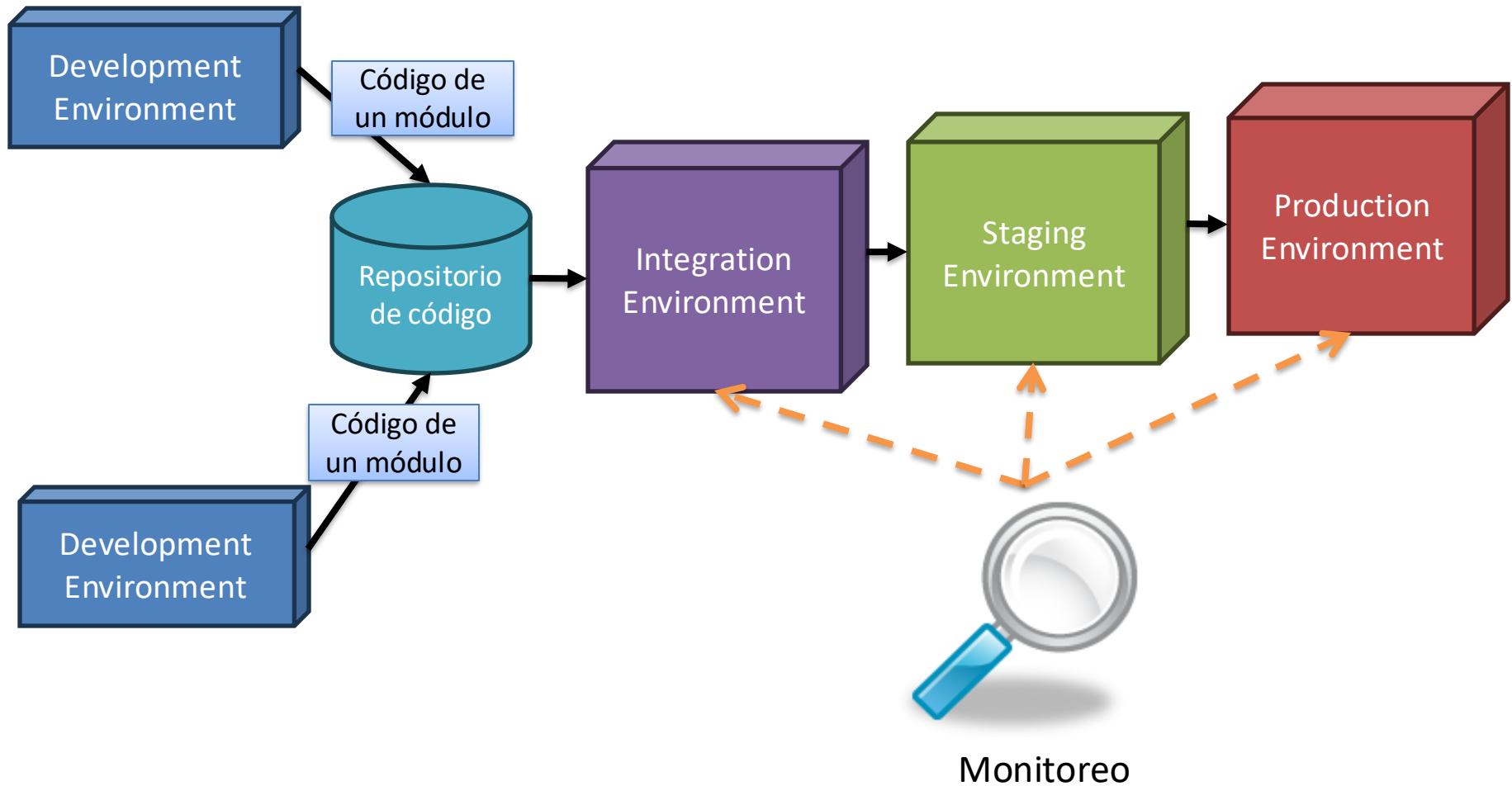
Release

- Entrega de software
- Versión estable
- Lista para ponerse en producción
- Frecuencia?
 - Antiguamente: Poco frecuente
 - Actualidad: Muy frecuente
 - Hotfixes
 - Adaptación a cambios

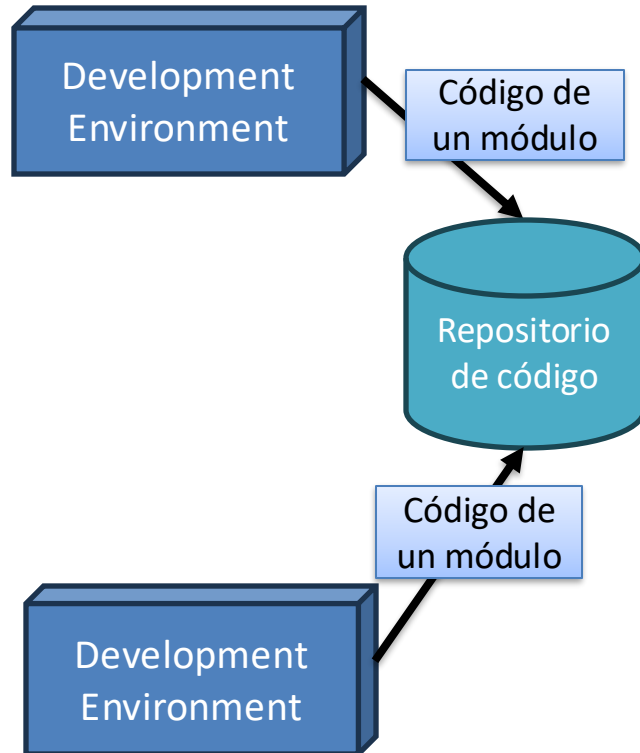
DevOps -> Continuous Integration + Continuous Deployment = CI/CD



CI/CD Pipeline



Development Environment



- Computador del desarrollador
 - IDE
 - Compiladores
 - Pruebas unitarias
 - Algunas pruebas de integración
 - Guarda el código del módulo en el repositorio

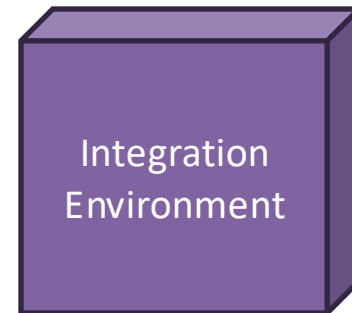
Repositorio de código

- Guarda el código de todos los módulos
- Control de versiones



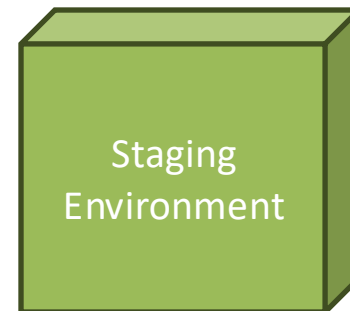
Integration Environment

- Servidor(es)
- Integra todos los módulos
 - Compila
 - Empaqueta
 - Ejecuta pruebas
 - Unitarias
 - Integración
 - Sistema



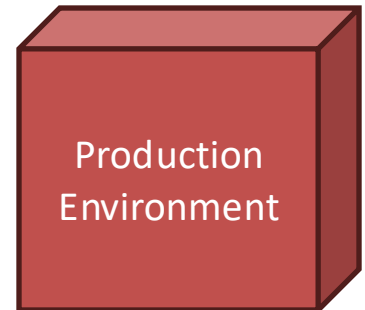
Staging Environment

- Servidor(es)
 - Ejecuta pruebas
 - Desempeño
 - Disponibilidad
 - Seguridad
 - Validación / Aceptación
 - Etc.



Production Environment

- Servidor(es)
 - Acceso a usuarios finales
 - Despliegue
 - Ejemplo:
 - Shadow → Monitoreo → Reintroducción



Monitoreo

- Todo lo que vimos sobre monitoreo en **Disponibilidad**



Monitoreo

Calidad de un Pipeline

- Tiempo de Ciclo ([Cycle Time](#))
 - Tiempo que demora en ponerse un requisito en producción (desde especificación hasta despliegue)
- Trazabilidad
 - Rastrear todos los elementos que dieron vida a un componente
 - Código fuente
 - Librerías
 - Casos de prueba
 - Herramientas
- Repetibilidad
 - Obtener los mismos resultados a partir de la misma acción sobre los mismos artefactos
 - "Pero si en mi computador funcionaba!"

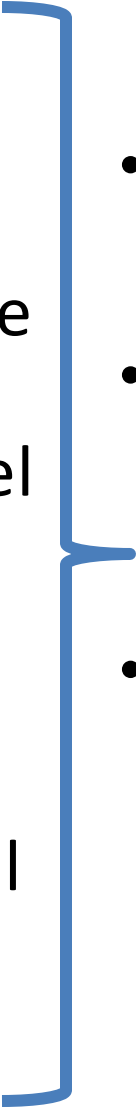
¿Qué es Deployability
(Desplegabilidad)?



Desplegabilidad

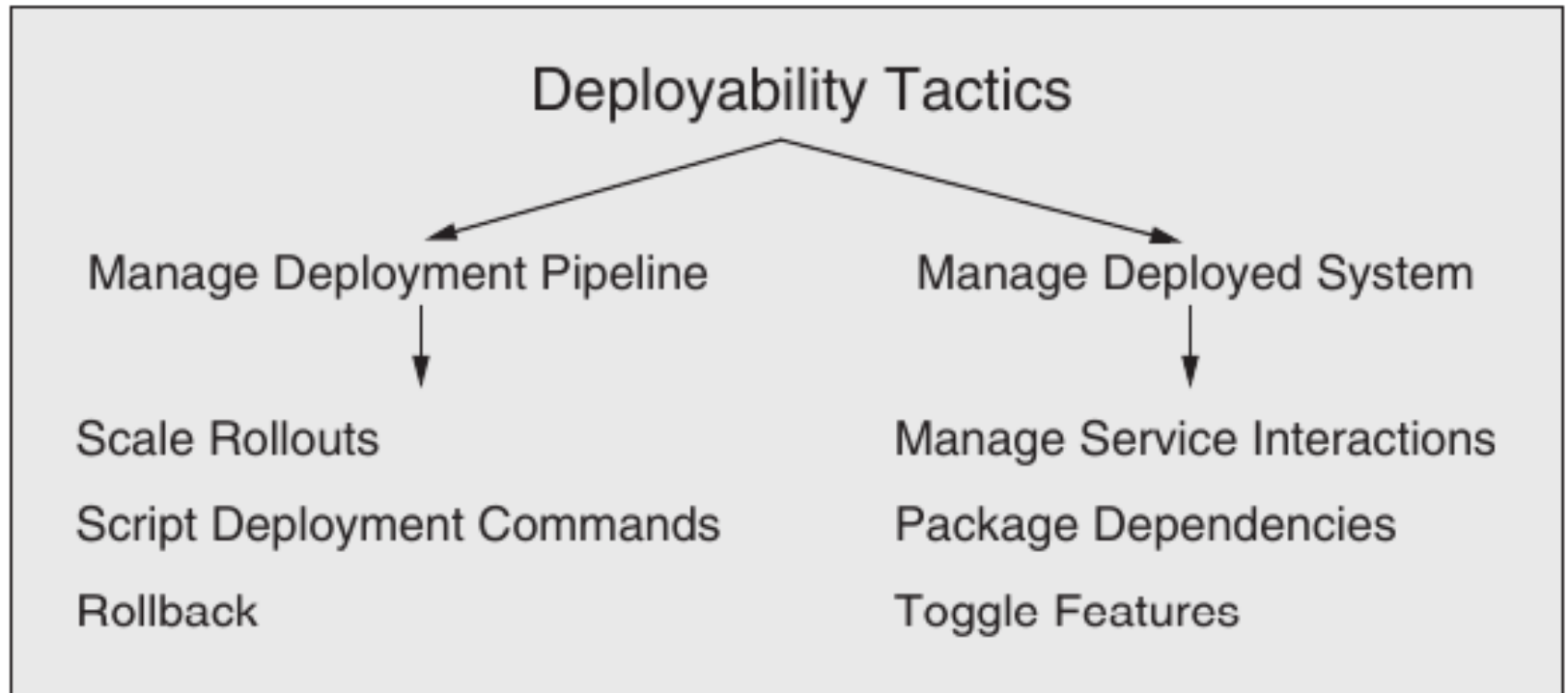
- Capacidad de un software de ser desplegado
 - Instalado en un ambiente para ejecución
 - Tiempo y esfuerzo razonables y predecibles

Problemas y preguntas

- 
- ¿Cómo se empaqueta?
 - ¿Cómo se envía el software al nodo donde se ejecutará?
 - ¿Cómo se integra con el resto del sistema?
 - ¿Qué tanto control sobre el proceso de despliegue?
 - ¿Qué tan eficiente es el despliegue?
- Granularidad
 - Componentes vs sistema completo
 - Controlabilidad
 - Cómo se despliega
 - Monitoreo del despliegue
 - Monitoreo de la ejecución
 - Eficiencia
 - Rapidez
 - Nivel de esfuerzo
 - Consumo de recursos

Tácticas de Desplegabilidad

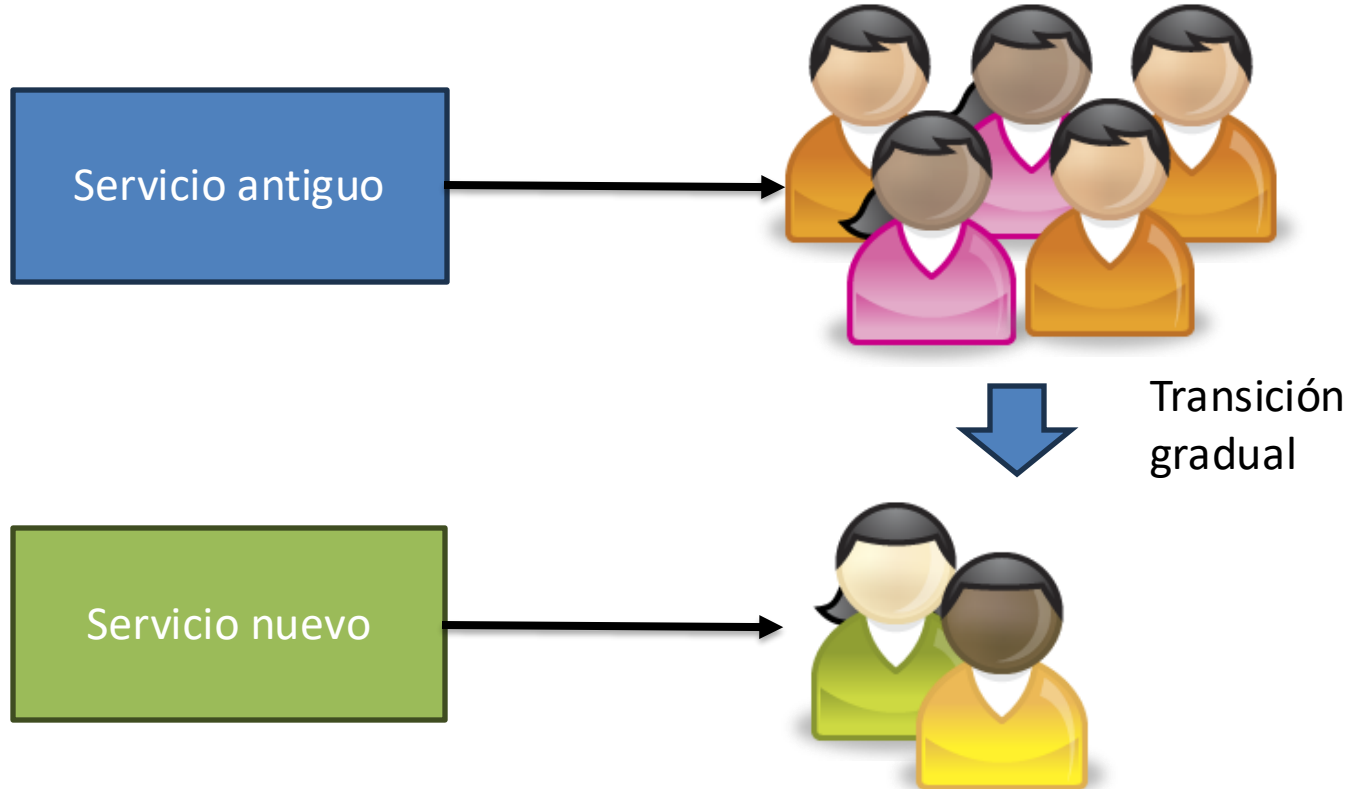
Tácticas de Desplegabilidad



Administración del pipeline de despliegue

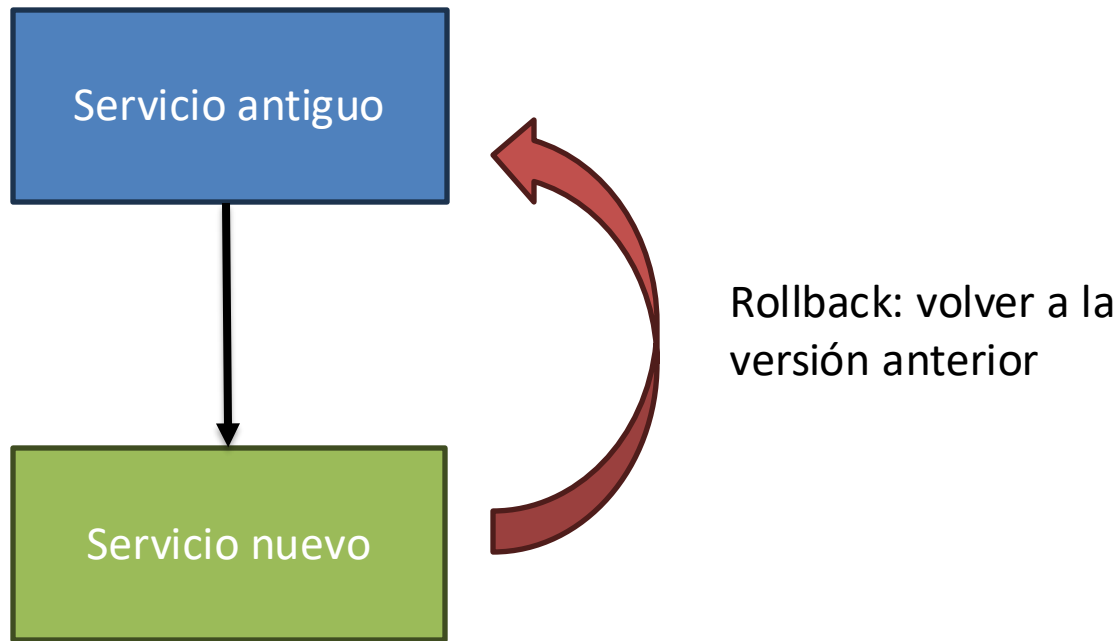
Scaled Rollouts

- Servicio nuevo se aplica a un subconjunto de los usuarios



Rollback

- Si el nuevo servicio falla, volver a la versión anterior



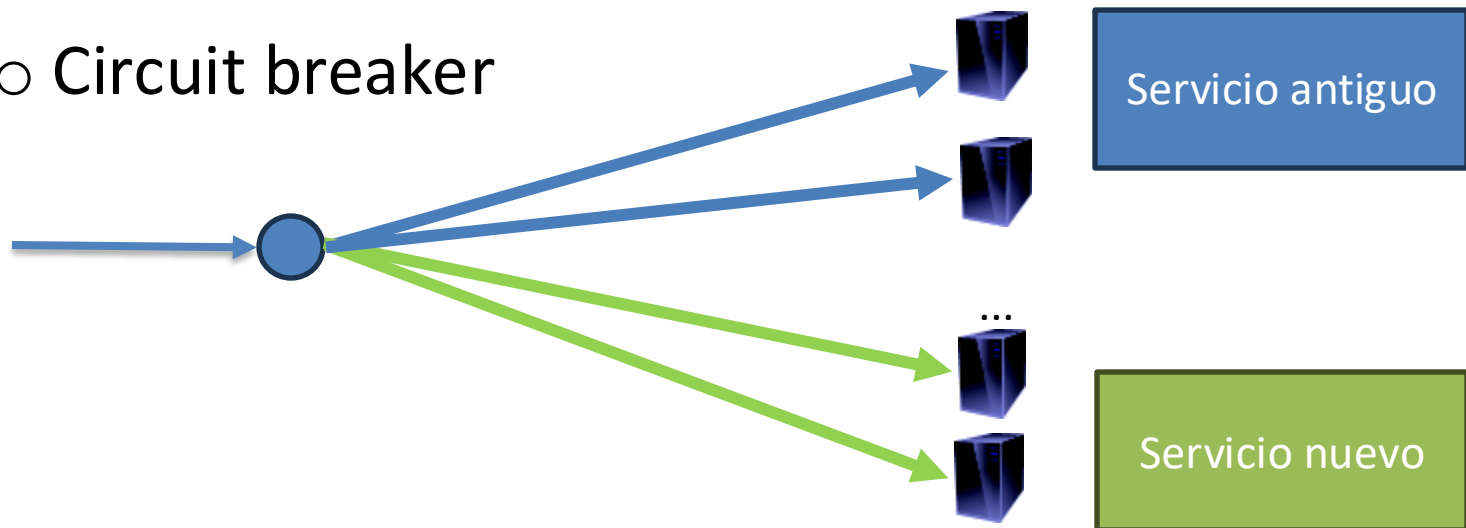
Scripts de despliegue

- En casa del herrero...
- ... se usan scripts para automatizar todo lo que se pueda automatizar
- Scripts de despliegue son un código fuente más en el sistema
 - Programar
 - Documentar
 - Revisión de código
 - Tesging
 - Control de versiones

Administración del sistema desplegado

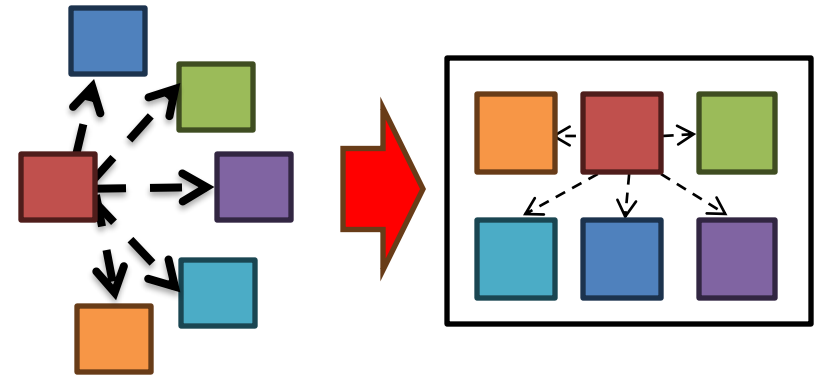
Administrar interacciones entre servicios

- Habilitar el despliegue de diferentes versiones de un servicio
 - Service registry
 - Traffic splitting
 - Load balancing
 - Circuit breaker



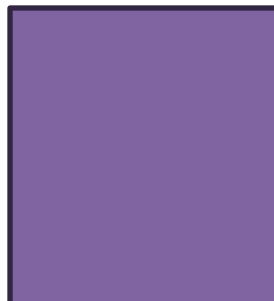
Empaquetar dependencias

- Dependencias
 - Librerías
 - Sistemas Operativos
 - Entornos de ejecución
- Empaquetar software junto con sus dependencias
 - Static linking
 - Contenedores
 - Pods (Kubernetes)
 - Máquinas virtuales



Activar/desactivar características

- "Kill Switch" para nuevas características
- Si fallan, se pueden desactivar características en tiempo de ejecución



Patrones para Desplegabilidad

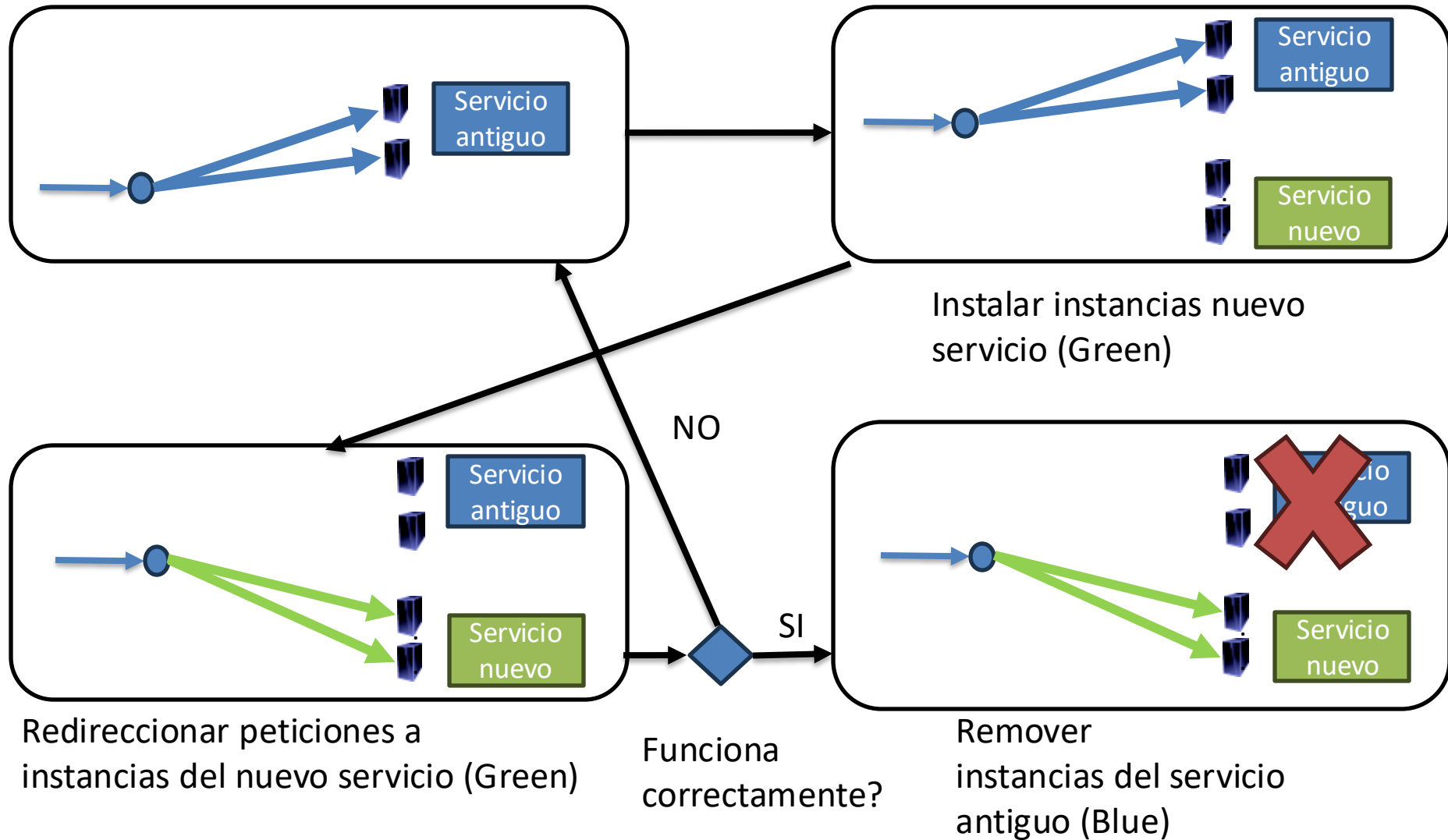
Patrones para Desplegabilidad

- Estructuración de Servicios
 - Desplegabilidad independiente
- Reemplazo total de servicios
 - Blue/Green
 - Rolling upgrade
- Reemplazo parcial de servicios
 - Canary testing
 - A/B Testing

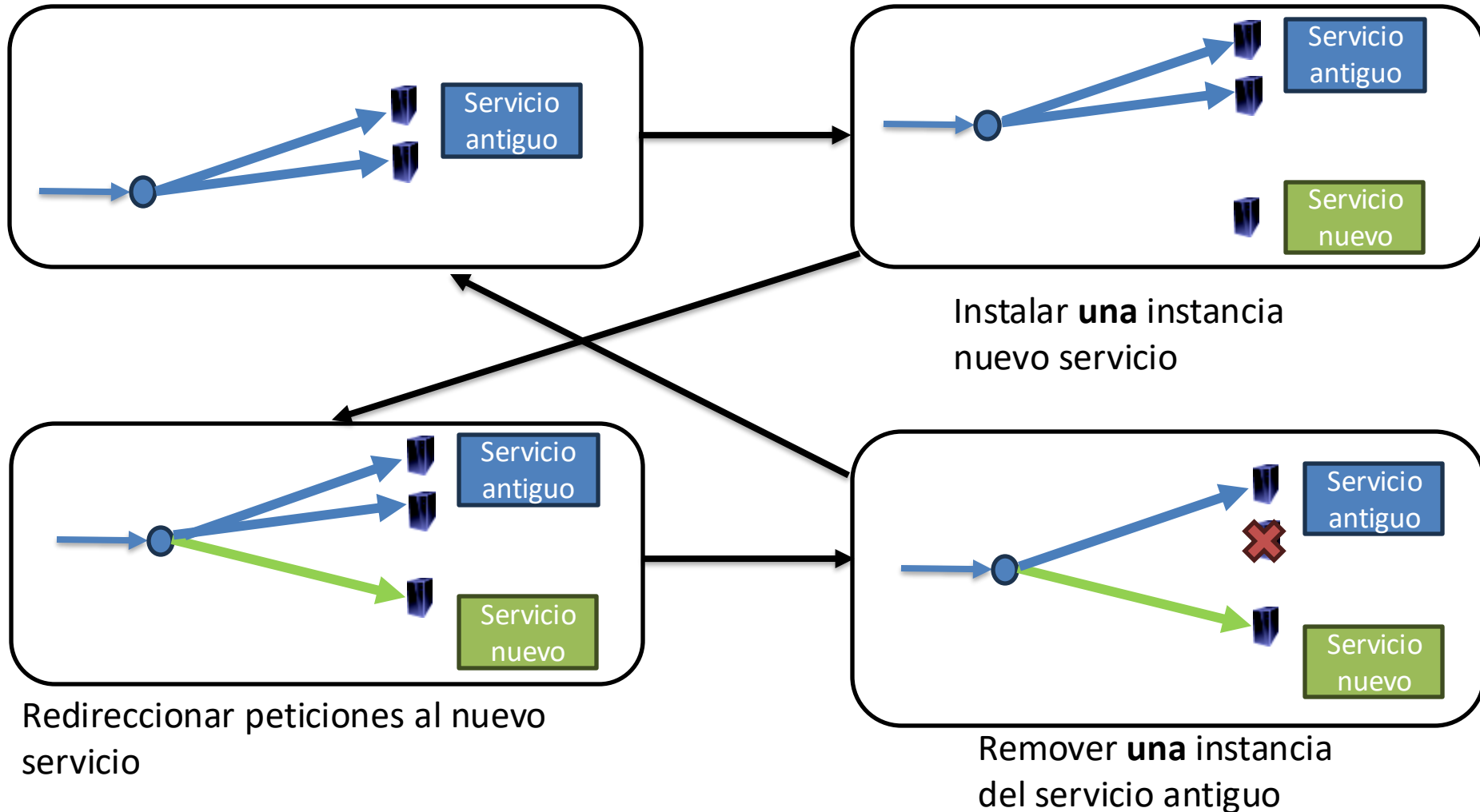
Desplegabilidad independiente

- Asegurar que cada componente(s) del software se pueda desplegar de manera autocontenida
 - ¿Cómo hacer un componente autocontenido?
 - Empaquetar dependencias
 - ¿Cómo comunicar componentes autocontenidos?
 - Administrar interacciones entre servicios

Blue/Green



Rolling Upgrade



Comparación

Blue/Green

- Mejora disponibilidad
- Requiere $2 \times N$ instancias
- Período explícito para buscar errores en el despliegue
- Menos probabilidad de inconsistencias

Rolling Upgrade

- Mejora disponibilidad
- Requiere $N + 1$ instancias
- Los errores se pueden detectar durante el upgrade
- Posibles inconsistencias
 - Inconsistencia temporal
 - Inconsistencia de interfaces

Canary Testing

- Designar un subconjunto de usuarios (Canarios)
 - Beta testers
 - Usuarios internos de la organización de software
 - Usuarios aleatorios
- Asignarles la nueva versión del servicio (Scaled rollout)
- Enrutamiento selectivo a servicios (usuarios normales vs canarios)
- Actualización final: Blue/Green o Rolling Upgrade

A/B Testing

- Sistema antiguo: A
- Sistema nuevo: B (ligeramente distinto a A)
- Se comparan métricas para determinar quien "gana"
 - Se descarta el que "pierde"

A/B Testing

Sistema antiguo (A)

- La mayoría sigue usando A

Sistema nuevo (B)

- Se le asigna a un subconjunto de usuarios
- Cambios menores en funcionalidad respecto de A

- Se comparan métricas sobre ambos sistemas
- Se escoge un "ganador"
- El "perdedor" se descarta



Comparación

Canary Testing

- Pruebas con usuarios reales
- Exposición reducida del sistema nuevo a usuarios finales
- Costos adicionales reducidos
 - Instancias nuevas
- Requiere planeación cuidadosa
- Requiere redireccionar selectivamente

A/B Testing

- Además de las características de Canary Testing:
 - Permite realizar experimentos sobre usuarios reales
 - Requiere un esfuerzo para implementar las nuevas características
 - Requiere un esfuerzo para caracterizar y seleccionar usuarios